Lesson 2 Teacher's Key

GDevelop

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Simplest path - just the behavior, no events needed. Level 1 only requires demonstrating the Right key, even though all 4 already work:

```
Object: Player
Behavior: Top-down movement (default settings)
```

Level 2 - Complete Worked Solution

No manual events needed - the Top-down movement behavior already reads all 4 arrow keys by default once applied to the Player object. Confirm this in Preview by testing all 4 directions.

```
Object: Player
Behavior: Top-down movement (default settings: 4-direction, Max speed ~150
px/s)
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

Flash visibility (blink) is an action that belongs to a behavior, not a general-purpose action - it won't appear in Player's action search until the Flash behavior is attached. Select Player > Behaviors tab > Add a behavior > search "Flash" > Apply. Only then do "Flash visibility (blink)" (start) and "Stop flashing visibility (blink)" (stop) show up when you search Player's actions - the same behavior-first pattern as Angle of movement needing Top-down movement in the Recolor idea below. No counter variable or Draw-event toggle needed, unlike the code-based engines:

```
Condition: Is moving  (Top-down movement)
Action: Flash visibility (blink) of Player, Duration 0 (keep flashing
until stopped)

Condition: Is moving = False
Action: Stop flashing visibility (blink) of Player
```

Sound: beep-beep while moving, silent when idle

Import sfx_blip.wav as a Sound resource. "Play a sound" fires once per call, so pairing it with a scene timer creates the beep-beep retrigger rhythm instead of one sound overlapping itself every frame:

```
At the beginning of the scene:
Action: Start (or reset) a scene timer "Blip"

Every frame:
Condition: Is moving  (Top-down movement)
Condition: Timer value "Blip" >= 0.3 seconds
Action: Play a sound (sfx_blip.wav)
Action: Start (or reset) a scene timer "Blip"
```

Recolor: different sprite per direction, reset when idle

Same Idle/Moving concept as the State Diagram from this lesson's bonus section. This swaps which Animation is showing rather than tinting the object - add the other 4 placeholder circles as extra Animations on Player (Edit object > Animations > Add an animation), one named Red, Blue, Green, and Yellow, plus your default one renamed Orange. The thing being built here is 5 separate Events, not conditions/actions on their own - each Event is one row that bundles together its own Condition(s) and Action(s), and an Event's Action(s) only run when all of that SAME Event's Condition(s) are true. There's no separate step to "wire" a condition to an action; they're never independent pieces, only ever parts of the one Event that owns them - which is also why cramming multiple direction-conditions into a single Event would AND them together instead of offering a choice between them. In the Scene's Events tab, each Event row has small, subdued text at its top-left reading Add condition and Add action - no icon, easy to skim past. For each of the 5 Events: click its Add condition, pick Player, then look for a separate Top-down movement category in the list (not Player's own conditions, since this one belongs to the behavior) and choose Angle of movement - it compares to a single angle plus a tolerance, so each direction needs its own Event rather than one shared condition. Then, still inside that same Event, click its Add action, pick Player again, and search for Animation (by name) under Animations and images - it's labeled that way in the picker, but displays as "Change the animation of Player..." once placed:

```
Event 1 - Condition: Angle of movement = 0 deg (tolerance 45)    Action:
Change the animation of Player to "Red"     (Right)
Event 2 - Condition: Angle of movement = 90 deg (tolerance 45)   Action:
Change the animation of Player to "Blue"    (Down)
Event 3 - Condition: Angle of movement = 180 deg (tolerance 45)  Action:
Change the animation of Player to "Green"   (Left)
Event 4 - Condition: Angle of movement = 270 deg (tolerance 45)  Action:
Change the animation of Player to "Yellow"  (Up)
Event 5 - Condition: Is moving = False                           Action:
Change the animation of Player to "Orange"  (idle)
```

Clamp movement to the screen

Add the Stay On Screen (2D) behavior to Player (Add a behavior > search "Stay On Screen"). It constrains the object's position to the camera viewport automatically, the same idea as GameMaker's clamp() or Godot's clamp() - just as a behavior instead of a line of logic, so no events are needed at all.

Structure: group everything triggered by the same action in one place

GDevelop's version of "one place in code" is an event group - right-click the event sheet, Add a group, then drag the Animate, Sound, and Recolor events (goals 1-3) inside it and label it something like "Direction feedback." The Stay On Screen behavior and the Level 1/2 movement events stay outside the group since they're not part of the same triggered action. That's the Complete Level 3 solution below.

Complete Level 3 solution (all 5 goals combined)

Everything above, organized as one event group named "Direction feedback":

```
At the beginning of the scene:
Action: Start (or reset) a scene timer "Blip"

Event group: "Direction feedback"

  Condition: Is moving  (Top-down movement)
  Action: Flash visibility (blink) of Player, Duration 0

  Condition: Is moving
  Condition: Timer value "Blip" >= 0.3 seconds
  Action: Play a sound (sfx_blip.wav)
  Action: Start (or reset) a scene timer "Blip"

  Condition: Angle of movement = 0 deg (tolerance 45)    ->  Action:
Change the animation of Player to "Red"     (Right)
  Condition: Angle of movement = 90 deg (tolerance 45)   ->  Action:
Change the animation of Player to "Blue"    (Down)
  Condition: Angle of movement = 180 deg (tolerance 45)  ->  Action:
Change the animation of Player to "Green"   (Left)
  Condition: Angle of movement = 270 deg (tolerance 45)  ->  Action:
Change the animation of Player to "Yellow"  (Up)

  Condition: Is moving = False
  Action: Stop flashing visibility (blink) of Player
  Action: Change the animation of Player to "Orange"

(Stay On Screen (2D) behavior on Player handles Clamp automatically - no
event needed)
```